

PROVISIONAL PATENT APPLICATION

**LIFECYCLE-MANAGED RECURSIVE
REPRESENTATION COMPUTING SYSTEMS AND
METHODS**

Inventor(s)	Mark White
Applicant / Assignee	Forma Substrate Inc
Prepared for	Forma Substrate Inc
Date	August 10, 2026
Status	Filed

Abstract

Systems and computer-implemented methods are disclosed for maintaining one or more recursively structured computational representations under continuing changes in observations, states, constraints, or relationships. A representation runtime maintains multiple representation views having different materialization and freshness states, tracks dependencies among identities, transformations, neighborhoods, recursive regions, hierarchy levels, paths, and diagnostics, and selectively invalidates affected portions when an upstream dependency changes. The runtime determines a recomputation scope, including local, regional, or global scope, and performs incremental re-indexing or other structural recomputation while preserving unaffected portions. Recursive hierarchy levels may be materialized, lazily materialized, virtualized, dematerialized, or refreshed according to structural need, freshness budget, dependency state, computational cost, resource availability, or query demand. The runtime may employ hysteresis, persistence windows, event coalescing, and separate escalation and recovery thresholds to reduce repeated rebuild oscillation. A mutable working representation may be frozen into a reproducible snapshot containing dependency closure, configuration, provenance, and structural state, and the snapshot may be published as an immutable representation product for independent replay or evaluation. The disclosed architecture separates execution-level maintenance from structural interpretation and scientific qualification and may operate with recursive indexing, graph, manifold, topological, state-transition, constraint-based, or other structural computation engines.

1. Technical Field

The present disclosure relates generally to computer architecture, representation systems, graph and relational computation, recursive indexing, incremental computation, runtime scheduling, dependency management, scientific computing, and machine-learning infrastructure.

More particularly, the disclosure concerns representation-native computing systems that maintain multiple recursively structured representations over changing observations or system states, selectively materialize structural hierarchy, track dependencies, invalidate affected regions, perform scoped incremental structural recomputation, manage representation freshness, and freeze mutable runtime state into reproducible snapshots.

2. Background

Modern computational systems commonly treat representations as transient intermediates. A dataset or event stream is converted into vectors, embeddings, graphs, clusters, state-transition models, or other derived objects; an algorithm is run; and the derived representation may be discarded or reconstructed when the underlying data changes. This workflow can be inefficient when the representation contains expensive relational, topological, recursive, or path-dependent structure.

For recursively structured representations, an apparently local change may affect neighborhoods, reachability relationships, recursive regions, higher-order aggregates, paths, hierarchy levels, and derived diagnostics. Reconstructing an entire representation after every change wastes computation, while leaving stale derived structure in place may cause downstream operations to act on invalid structural state.

Existing graph-processing, database, machine-learning, and caching systems provide mechanisms for incremental computation, lazy evaluation, view maintenance, and cache invalidation. A need remains for an architecture specifically adapted to recursively structured computational representations in which identity, relationship, admissibility, neighborhood, path, recursive hierarchy, scale, and derived structural state are interdependent.

A further problem arises when a live mutable representation is also used as evidence for scientific, regulatory, engineering, or high-consequence evaluation. A mutable runtime object is useful for ongoing computation, but evaluation evidence should be reproducible and traceable. There is therefore a need for a controlled boundary between mutable working state and a frozen representation snapshot whose dependencies, configuration, lineage, and structural state can be independently replayed.

The inventors recognized that representation lifecycle, freshness, dependency, materialization, recomputation scope, and publication state can be treated as first-class runtime properties rather than hidden implementation details.

3. Summary of the Disclosure

In one aspect, a representation runtime maintains a plurality of representation views derived from shared observations, states, or source objects. Each view includes lifecycle metadata identifying at least a materialization state, freshness state, dependency set, update state, and one or more structural scopes or scales.

In another aspect, the runtime maintains a dependency structure linking source objects to derived structural objects. When a source object or upstream structural object changes, the runtime identifies a dependency closure, marks affected objects or regions as dirty or stale, preserves unaffected regions, and schedules recomputation only for the affected scope unless a policy requires broader reconstruction.

In another aspect, the runtime supports local, regional, and global recomputation scopes. Scope may be selected based on dependency footprint, structural impact, affected hierarchy level, change magnitude, resource budget, freshness requirement, correctness policy, or structural-health measurements.

In another aspect, recursive hierarchy levels are not required to remain fully materialized. Lower levels may remain current while higher levels remain lazy or virtual and are materialized in response to queries, structural events, freshness requirements, scheduled refreshes, or resource policies.

In another aspect, secondary views are permitted bounded staleness. A secondary view may remain stale while a primary view remains current, provided that one or more configured conditions are satisfied. The runtime may prevent use of a stale view for selected operations or downgrade the view to an invalidated state when a freshness budget is exceeded.

In another aspect, the runtime separates computational events from structural interpretation and from scientific or operational qualification. A low-level event may identify a dirty region; a structural engine may interpret recomputed structure; and a separate evaluation subsystem may determine what conclusions or actions are permitted.

In another aspect, the runtime uses hysteresis, persistence windows, dwell times, event coalescing, priority rules, or distinct enter/exit thresholds to reduce oscillatory rebuild behavior.

In another aspect, a mutable working representation is frozen into a snapshot containing structural objects, dependency closure, configuration, source references, software/environment identifiers, seeds where relevant, freshness state, and provenance. The snapshot can be registered as an immutable representation product for independent replay, comparison, or qualification.

Core inventive combination: The disclosure focuses on the coordinated use of recursively structured representations, explicit lifecycle states, dependency-governed invalidation, selective hierarchy materialization, bounded staleness, scoped incremental re-indexing, anti-thrashing control, and reproducible snapshot publication. The invention is not limited to a particular graph algorithm, database, or RIX implementation.

4. Brief Description of the Drawings

FIG. 1. Layered architecture showing a representation runtime beneath a structural computation engine and above numerical compute resources.

FIG. 2. Representation-view object model including lifecycle, freshness, dependency, scale, and provenance metadata.

FIG. 3. Materialization states including primary, materialized, lazy, and virtual.

FIG. 4. Freshness lifecycle including current, stale-permitted, refresh-required, and invalidated.

FIG. 5. Dependency graph linking source observations to identities, relationships, neighborhoods, recursive regions, paths, hierarchy levels, and diagnostics.

FIG. 6. Event-driven structural invalidation process.

FIG. 7. Local, regional, and global recomputation scopes.

FIG. 8. Selective recursive-hierarchy materialization process.

FIG. 9. Anti-thrashing state machine using persistence and distinct enter/exit thresholds.

FIG. 10. Working-to-snapshot-to-published lifecycle.

FIG. 11. Execution contract between a structural computation engine and representation runtime.

FIG. 12. Distributed implementation with CPU/GPU kernels, scheduler, dependency service, representation store, and event service.

FIG. 13. Incremental-update equivalence test comparing scoped recomputation with a full rebuild.

FIG. 14. Multiple representation views sharing a common source observation set.

5. Definitions and Interpretive Conventions

Term	Meaning
Observation or source object	Any datum, state, record, event, sample, sequence, embedding, latent state, sensor observation, simulation state, symbol, biological sequence, physical measurement, financial observation, machine state, or other computationally represented input.
Identity	A distinguishable computational entity, state, address, element, node, token, residue, event, observation, region, or higher-order object that can participate in relationships or transformations.
Transformation	A relationship, transition, mapping, operation, admissible move, edge, rule, constraint-conditioned transition, or other relation between identities or structural objects.

Representation	A computational organization of identities, transformations, relationships, coordinates, neighborhoods, paths, hierarchy, constraints, or derived structure.
Representation view	A particular computational realization or structural view of common or related observations. Different views may use different construction rules, scales, materialization policies, or structural operators.
Recursive structural representation	A representation in which structures generated at one level can become inputs, identities, regions, aggregates, or constraints at another level.
Structural index	A data structure or object encoding neighborhoods, regions, basins, recursive aggregates, paths, reachability, topology, hierarchy, or other organization.
Materialized	Computed and stored for direct use without reconstruction from its generator or dependencies.
Virtual	Represented by construction metadata, generator definitions, dependencies, or references sufficient to generate the structure when required, without a complete stored materialization.
Lazy	Not presently current or materialized, but eligible for generation upon demand or a trigger.
Freshness budget	A rule defining how much time, source update, structural change, dependency divergence, error, or deviation may occur before refresh or invalidation.
Dirty	A state indicating that an object may no longer be consistent with changed dependencies and requires evaluation or recomputation before selected uses.
Dependency closure	The downstream set of objects whose values or structural state may depend directly or indirectly on a changed source or derived object.
Re-indexing	Recomputation, reconstruction, repair, updating, or replacement of one or more structural indices or recursively derived objects.
Local scope	A recomputation scope limited to an affected

	identity, neighborhood, or bounded dependency region.
Regional scope	A recomputation scope covering a basin, component, hierarchy branch, adjacent regions, or other bounded subrepresentation.
Global scope	A recomputation scope covering substantially all of a representation view or hierarchy.
Snapshot	A frozen computational state sufficient to reproduce, replay, inspect, compare, or independently evaluate a representation at a selected point.
Published representation product	An immutable or append-only representation artifact derived from a snapshot and associated with provenance. Publication can be internal or external.
Structural engine	A computation engine that determines or interprets structural organization. Recursive Indexing is one non-limiting embodiment.
Representation runtime	The execution subsystem that manages lifecycle, dependencies, materialization, freshness, recomputation scheduling, resource policy, and snapshotting. Primitive Indexing is one non-limiting embodiment.

6. Overall System Architecture

```

Application / scientific / operational subsystem
|
Independent evaluation or qualification subsystem
|
Representation engineering subsystem
|
Structural computation engine
(recursive indexing / graph / topology / manifold / state transition)
|
Representation runtime
lifecycle | dependencies | freshness | materialization | scheduling | snapshots
|
Numerical / storage substrate
CPU | GPU | sparse kernels | databases | object storage | distributed compute

```

The layers may be deployed in a single process, multiple processes, one machine, multiple machines, a cloud environment, an embedded device, an autonomous system, a scientific cluster, or combinations thereof. The runtime may be implemented as a library, service, operating-system subsystem, middleware layer, accelerator runtime, database extension, graph-processing subsystem, or distributed execution service.

In a preferred embodiment, the structural engine specifies what structural operation is required and the representation runtime determines how to execute and maintain the required state. A separate evaluation subsystem may inspect frozen outputs without modifying them.

7. Representation Runtime Object Model

7.1 RepresentationView

```
RepresentationView {
    view_id
    source_ids[]
    construction_or_generator_id
    materialization_state
    freshness_state
    freshness_budget
    structural_scales[]
    dependency_refs[]
    dirty_regions[]
    update_epoch
    resource_profile
    policy_profile
    provenance_refs[]
    snapshot_refs[]
}
```

A view may be a coordinate representation, graph representation, recursive index, path or transport representation, hierarchical representation, compressed representation, multiscale representation, compatibility-weighted representation, or other structural view.

7.2 Identity and Transformation Objects

```
Identity { id, source_ref, type, attributes, version, dependency_refs }
Transformation { source_id, target_id, constraint_ref, cost, admissibility, scale,
version }
```

7.3 StructuralIndex

```
StructuralIndex {
    index_id
    view_id
    index_type
    scale_id
    member_refs[]
    adjacency_or_relation_data
    parent_index_ref
    child_index_refs[]
    dependency_refs[]
    materialization_state
    freshness_state
    structural_hash
}
```

8. Representation Lifecycle and Materialization

8.1 Materialization classes

PRIMARY: View maintained under the strongest currentness requirement for a selected workflow.

MATERIALIZED: Computed secondary view available for use, optionally under bounded staleness.

LAZY: Construction deferred until demand, scheduled refresh, or trigger.

VIRTUAL: Represented by generator/dependency metadata without complete current storage.

8.2 Freshness states

- **CURRENT:** relevant dependencies incorporated according to policy.
- **STALE-PERMITTED:** one or more updates not yet incorporated, but the view is still usable for selected operations.
- **REFRESH-REQUIRED:** refresh must occur before designated operations.
- **INVALIDATED:** designated use is prohibited until repair, recomputation, replacement, or re-establishment.

8.3 Freshness budget embodiments

- Time-based budget.
- Update-count budget.
- Structural-change budget based on neighborhood membership, edge set, reachability, path cost, region assignment, or structural health.
- Resource-aware budget.
- Use-aware budget in which exploratory use tolerates more staleness than safety-critical, scientific, or regulatory use.
- Hierarchy-aware budget in which lower levels stay current while higher levels lag.

9. Dependency-Governed Structural Invalidation

```
Source Observation
-> Identity / feature object
-> local relationships
    -> neighborhood index
        -> recursive region / basin
            -> path / transport index
                -> hierarchy level
                    -> structural diagnostic
```

The dependency structure may be stored as a graph, hypergraph, table, lineage map, event subscription structure, version vector, Merkle-style tree, or other data structure.

9.1 Change processing method

1. Receive a change identifying one or more changed source or derived objects.
2. Determine direct dependencies.
3. Expand to a bounded or complete downstream dependency closure.
4. Estimate structural impact and affected regions.
5. Mark affected objects dirty, stale, refresh-required, or invalidated.
6. Preserve unaffected objects and materializations.
7. Schedule incremental recomputation.
8. Update versions, hashes, freshness states, dependencies, and event records.

9. Propagate additional invalidation upward only when recomputed change satisfies a configured condition.

9.2 Structural cache coherence

Derived structural state can be treated as a representation cache. Each derived object may record the upstream versions that contributed to it. Before use, the runtime may compare dependency versions, hashes, freshness state, or epoch values. A mismatch may trigger refresh, use restriction, asynchronous repair, or substitution with another current view.

10. Scoped Incremental Re-Indexing

Scope	Illustrative boundary	Typical trigger	Goal
LOCAL	Identity, neighborhood, bounded dependency region	Small update	Repair minimal state
REGIONAL	Basin, component, hierarchy branch, adjacent regions	Boundary or path change	Restore coherent region
GLOBAL	Whole representation or hierarchy	Global constraint change or repeated repair failure	Re-establish complete coherence

10.1 Scope-selection policy

$$Score(s) = w1 * C_{dependency}(s) + w2 * C_{compute}(s) + w3 * C_{latency}(s) + w4 * C_{risk}(s) + w5 * C_{freshness}(s) + w6 * C_{structural}(s)$$

A policy may select the lowest-cost scope satisfying correctness and freshness requirements. Deterministic rules, heuristics, optimization, learned policies, or hybrid methods may be used.

10.2 Equivalence and correctness checks

An incrementally maintained representation may periodically be compared with a complete rebuild using exact equality, neighborhood overlap, graph edit distance, path-cost agreement, region agreement, hierarchy consistency, hashes, numerical tolerance, or domain-specific structural criteria. Excess divergence can trigger broader future scope or a global rebuild.

11. Recursive Hierarchy and Selective Materialization

Level 0	source-linked identities	MATERIALIZED / CURRENT
Level 1	local neighborhoods	MATERIALIZED / CURRENT
Level 2	recursive regions or basins	MATERIALIZED or LAZY
Level 3	sub- or super-regions	LAZY
Level 4+	deeper hierarchy	VIRTUAL by default

Any level may be materialized or virtual. Policy may consider query frequency, dependency-change frequency, compute cost, memory pressure, expected reuse, decision importance, structural instability, scale, or user request.

11.1 Materialization triggers

- Query requiring an unmaterialized level.
- Structural change not resolvable at currently materialized levels.

- Scheduled refresh or prefetch.
- Anticipated downstream operation or replay request.
- Improved resource availability.
- Structural event indicating that additional scale is required.

11.2 Dematerialization

A region may be dematerialized when recreation cost is acceptable, reuse is low, resource pressure is high, a newer parent supersedes it, or policy permits reconstruction. Generator metadata, provenance, and dependencies can be retained to permit reconstruction.

12. Layered Event Architecture

```

Runtime event
  DEPENDENCY_CHANGED / VIEW_STALE / REGION_DIRTY / MATERIALIZATION_REQUIRED
  |
Structural event
  NEIGHBORHOOD_CHANGED / REGION_SPLIT / CONNECTIVITY_LOSS / PATH_REORGANIZED
  |
Evaluation event
  REGIME_TRANSITION_EVIDENCE / QUALIFICATION_CHANGE / ACTION_RESTRICTION

```

A runtime event need not imply a structural or scientific condition. A structural event may be preserved as evidence without automatically causing an operational decision. This separation permits independent replay and governance.

12.1 Event payload

```

StructuralRuntimeEvent { event_id, event_type, source_refs[], affected_views[],
  affected_regions[], pre_versions[], post_versions[], severity, freshness_effect,
  requested_scope, epoch, provenance_ref }

```

12.2 Event coalescing

Events occurring within a configured interval or affecting overlapping regions may be coalesced into a shared recomputation task. Coalescing can be disabled for urgent events or event-by-event replay.

13. Hysteresis and Anti-Thrashing

- Distinct entry and exit thresholds.
- Minimum dwell time.
- Persistence across k epochs.
- Accumulated-change thresholds.
- Cooldown after global rebuild.
- Backoff after repeated local repair failure.
- Event priority and suppression rules.
- Resource-sensitive hysteresis.

For example, a view may become STALE-PERMITTED after one unresolved update, REFRESH-REQUIRED after three, and INVALIDATED after a structural threshold is exceeded. Recovery can require successful recomputation plus multiple stable epochs. These values are illustrative.

14. Working, Snapshot, and Published Representation States

14.1 Working state

A working representation is mutable and may accept observations, update dependencies, create or retire views, change materialization, recompute indices, and update runtime health.

14.2 Freeze operation

10. Acquire a consistent logical epoch or transaction boundary.
11. Identify views included in the snapshot.
12. Resolve or record permitted stale dependencies.
13. Capture construction/generator identifiers and parameters.
14. Capture source references or cryptographic hashes.
15. Capture structural-index versions and hashes.
16. Capture dependency closure or reproducible dependency manifest.
17. Capture software, library, model, and execution-environment versions where relevant.
18. Capture random seeds or deterministic replay state where relevant.
19. Capture materialization and freshness states.
20. Generate a snapshot identifier and integrity hash.
21. Prevent mutation of the snapshot or create a new version on later change.

14.3 Published product

A snapshot may be registered as an immutable representation product in a repository, evaluation system, scientific instrument, regulatory record, production registry, or audit system. Later working-state changes produce later snapshots rather than modifying the published object.

```
WORKING (mutable) --freeze--> SNAPSHOT (frozen) --register/publish-->
REPRESENTATION PRODUCT (immutable or append-only)
```

15. Runtime Structural Health

The runtime may maintain computational health measurements for scheduling, invalidation, or scope selection. These measurements do not themselves establish scientific validity or entitlement.

$RSV(R) = [N, T, I, S]$

- N: neighborhood preservation or local relational coherence.
- T: transport/path preservation, connectivity, or reachability robustness.
- I: integrity measurements reflecting sensitivity to decomposition, perturbation, reconstruction, or dependency loss.
- S: scale consistency across selected recursive levels.

Components may be used independently. A universal scalar is not required.

16. Structural Engine ↔ Representation Runtime Contract

```
Structural request:
operation = REINDEX
target_view = R
structural_scope = region_27
scale = 16
correctness_policy = STRICT
```

```

Runtime actions:
  resolve dependencies
  materialize prerequisites
  invalidate affected objects
  choose execution resources
  recompute selected scope
  propagate required updates
  update freshness/version state
  emit completion/result event

```

Other operations may include construct, update, attach observation, detach observation, compute neighborhood, compute region, compute path, refresh view, materialize scale, dematerialize scale, compare representations, freeze snapshot, or validate incremental equivalence.

17. Example Programming Model

```

runtime = RepresentationRuntime(policy=policy)
source = runtime.register_source(observations)

direct = runtime.create_view(source, generator=DirectGenerator(), mode="PRIMARY")
recursive = runtime.create_view(direct, generator=RecursiveGenerator(scale=32),
mode="LAZY")

runtime.apply_update(source, new_observation)
runtime.ensure_current(recursive, purpose="structural_query")
state = runtime.state_vector(recursive)
snapshot = runtime.freeze([direct, recursive], include_dependencies=True)

```

The programming model may be exposed as ordinary typed objects, service APIs, a query language, or another interface. API names are non-limiting.

18. Detailed Algorithmic Embodiments

18.1 Incremental change propagation

```

INPUT: changed object c, representation view R, dependency graph G
1. increment version(c)
2. Q <- direct_dependents(c, G)
3. affected <- empty
4. while Q not empty:
    x <- pop(Q)
    if impact_rule(c,x): mark x dirty; add x to affected; enqueue eligible
dependents(x,G)
5. partition affected into structural regions
6. choose scope for each region
7. schedule recomputation
8. update versions/hashes/freshness
9. propagate only material post-recompute changes
OUTPUT: updated R and event record

```

18.2 Scope escalation

```

attempt LOCAL repair
if correctness check passes: commit
else attempt REGIONAL repair
if correctness check passes: commit

```

```
else perform GLOBAL rebuild
record escalation path and cause
```

18.3 Lazy hierarchy resolution

```
on request for hierarchy level L:
  if L is CURRENT and MATERIALIZED: return L
  if L is STALE-PERMITTED and policy allows stale: return L with freshness metadata
  determine prerequisites P
  materialize/refresh missing prerequisites
  compute L
  cache L according to policy
  return L
```

18.4 Snapshot freeze

```
begin logical freeze epoch e
version-pin included source/dependency objects
resolve snapshot freshness policy
capture view manifests, versions, generators, parameters, dependencies, hashes
capture environment/random-state metadata as required
compute snapshot integrity identifier
release pins
return immutable snapshot
```

19. Numerical, Storage, and Distributed Implementations

Dense numerical operations may be delegated to CPU, GPU, TPU, vector processor, or accelerator libraries while the runtime retains lifecycle/dependency control.

Neighborhood operations may use exact or approximate nearest neighbors, adjacency lists, sparse structures, graph databases, vector databases, distributed key-value stores, or custom in-memory structures.

Path and reachability operations may use shortest-path, dynamic graph, transport, message-passing, or domain-specific algorithms.

Distributed systems may partition by structural region rather than raw position and may track cross-partition dependencies.

Consistency may be strong, eventual, bounded-staleness, snapshot-isolated, or policy dependent.

Representation views may reside in memory, persistent storage, databases, object stores, accelerator memory, shared memory, or combinations thereof.

20. Recursive Indexing (RIX) Embodiment

In one embodiment, the structural engine performs Recursive Indexing (RIX). A source representation is converted into one or more structural views containing neighborhoods, compatibility relationships, recursive regions or basins, path or transport relationships, landmarks, or higher-order descriptors. The runtime does not define the mathematical meaning of those structures; it manages their construction state, dependencies, materialization, freshness, incremental update, and reproducibility.

When one observation changes, the runtime may identify affected direct coordinates, affected relationships, neighborhood indices, recursive regions, and higher structures, then refresh only the affected region. A local boundary change may escalate to regional repair. A global construction-parameter or constraint change may trigger global invalidation.

RIX is non-limiting. The runtime principles apply to hierarchical graph clustering, topological representations, manifold models, Markov-state models, finite-state systems, causal/relational representations, knowledge graphs, dynamic networks, biological landscapes, world-model latent spaces, robotics state spaces, financial regimes, industrial-process states, and other structured representations.

21. Independent Evaluation and Scientific-Object Embodiment

A live representation can remain mutable during an investigation. When ready for independent evaluation, the runtime freezes the exact relevant state into a snapshot. An evaluation subsystem receives the snapshot or a published immutable representation product rather than the mutable live object. The evaluation subsystem may replay, qualify, compare, or archive the product without modifying it. The runtime itself does not determine scientific entitlement or validity.

22. Example Applications

Machine-learning representation monitoring: Maintain direct, latent, graph, transport, or recursive views while new inputs arrive.

World models and robotics: Maintain organizational maps of latent/operating states and escalate re-indexing after persistent change.

Protein and biological landscapes: Maintain sequence, mutation, biochemical, alignment, neighborhood, and transport views while variants or references change.

Industrial process monitoring: Maintain structural regions while sensor observations stream.

Financial systems: Maintain organizational views with different freshness policies for exploratory and operational use.

Scientific instruments: Maintain live atlases and freeze reproducible products for independent analysis or publication.

Cyber-physical systems: Maintain structural state and restrict stale representations for selected control decisions.

23. Example Architecture Qualification Tests

- Incremental-vs-full rebuild equivalence.
- Controlled staleness-budget test.
- Local/regional/global scoped re-indexing comparison.
- Full versus lazy/virtual hierarchy-materialization comparison.
- Dependency invalidation completeness and selectivity test.
- Anti-thrashing test under oscillating metrics.
- Independent snapshot replay and agreement test.

24. Alternative Embodiments and Variations

- A view may depend on multiple datasets or another view.
- Dependencies may be exact or conservative.
- Freshness may be scalar, vector-valued, bounded error, or categorical.
- Materialization may occur at whole-view, level, region, object, edge, path, or tile granularity.

- Scope may be selected deterministically, heuristically, by optimization, by learned policy, or by operator request.
- Different views may use different structural engines.
- A snapshot may include stale-permitted secondary views when the manifest records that state.
- Immutable products may be content-addressed or version-addressed.
- Shared immutable views may support experiment-specific mutable overlays.
- Operations may execute on graph accelerators, sparse processors, FPGA, ASIC, near-memory devices, or future representation-native processors.

25. Technical Advantages

- Reduces unnecessary global recomputation.
- Permits multiple views without keeping every view fully live.
- Makes staleness explicit and policy controlled.
- Manages recursive hierarchy depth according to operational value and cost.
- Separates structural intent from runtime execution.
- Separates mutable computation from immutable evidence.
- Reduces rebuild thrashing.
- Improves replayability and provenance.
- Interoperates with existing numerical/tensor computing.

26. Illustrative Claim-Style Invention Statements

1. A computer-implemented method comprising maintaining a plurality of representation views derived from one or more source objects, each representation view having a materialization state and a freshness state; receiving a change to a source or derived object; identifying a dependency closure of structural objects affected by the change; invalidating fewer than all structural objects of at least one representation view; determining a recomputation scope; and incrementally recomputing structural objects within the determined scope while preserving unaffected structural objects.
2. The method wherein the structural objects comprise recursively derived neighborhoods, regions, basins, paths, reachability relationships, hierarchy levels, or combinations thereof.
3. The method wherein the recomputation scope is selected from local, regional, and global scopes.
4. The method wherein scope is based on dependency coverage, structural impact, compute cost, freshness, resource availability, correctness tolerance, or risk policy.
5. A system configured to maintain a primary representation view and a secondary representation view under different freshness budgets.
6. The system wherein the secondary view is usable while stale for a first operation and prohibited while stale for a second operation.
7. A method for selectively materializing a recursive hierarchy by keeping a first level materialized and a second level lazy or virtual, then materializing the second level in response to a query, event, freshness condition, or resource policy.
8. A method for representation cache invalidation comprising storing version-linked dependencies between source objects and recursively derived structural objects, detecting a version change, marking a dependent region dirty, and repairing the dependent region without reconstructing an unaffected region.

9. A representation runtime comprising a dependency service, representation store, materialization service, recomputation scheduler, and event service configured to perform dependency-governed incremental maintenance of a recursively structured representation.
10. A method comprising receiving from a structural computation engine a request identifying a structural operation, target representation, and structural scope; resolving dependencies and prerequisites; selecting execution resources; performing the operation; updating lifecycle metadata; and returning a result event.
11. A method applying separate transition thresholds, persistence requirements, or dwell periods to entry into and recovery from a representation state to reduce repeated structural recomputation.
12. A method coalescing invalidation events affecting overlapping dependency regions into a shared recomputation task.
13. A method freezing a mutable representation into a snapshot by capturing a consistent logical epoch, structural-object versions, dependency closure, construction configuration, source references, and provenance, and preventing later mutation of the snapshot.
14. The method further comprising registering the snapshot as an immutable representation product for independent replay or evaluation.
15. A computer-readable medium storing instructions that cause processors to perform any method described herein.
16. A distributed system partitioning structural regions among compute nodes, wherein cross-region dependency metadata controls invalidation and recomputation propagation.
17. A system wherein a structural health measurement selects among local, regional, and global recomputation without itself constituting scientific qualification.
18. A system wherein multiple views share a source set but have different generators, scales, freshness budgets, or materialization policies.
19. A method dematerializing a recursively derived object while retaining a generator definition and dependency information sufficient for reconstruction on demand.
20. A method periodically comparing an incrementally maintained representation with a full rebuild and modifying future recomputation policy based on structural divergence between them.

27. Relationship to Existing Portfolio and New Matter

This draft is intended to complement earlier subject matter relating to constraint-governed representations, constraint-derived geometry, recursive indexing, lawful transformational substrates, recursive computational compression, computational entropy, coordinate-structure divergence, and structural excursion/recovery. Those concepts may supply structural operators, objects, diagnostics, or use cases for embodiments of this runtime architecture.

- Multi-view lifecycle states and materialization classes.
- Freshness budgets and bounded staleness.
- Selective hierarchy materialization and virtualization.
- Dependency-driven structural invalidation.
- Local/regional/global scoped structural recomputation.
- Layered runtime-to-structural event contracts.
- Anti-thrashing and hysteresis for representation maintenance.
- Mutable working state to frozen snapshot to immutable published-product lifecycle.
- Runtime orchestration alongside conventional CPU/GPU/tensor computation.

Appendix A — Draft Figure Schematics

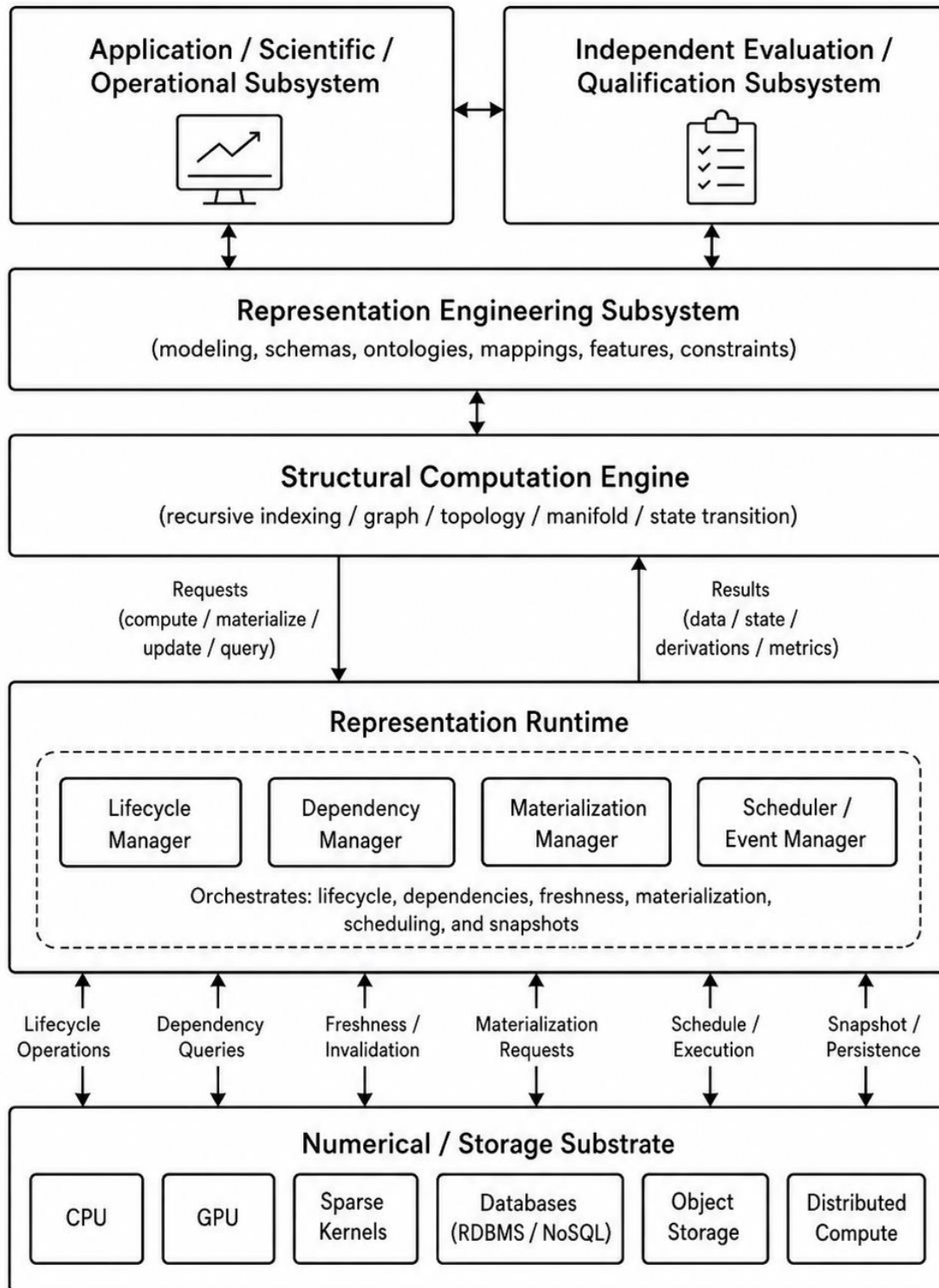


FIG. 1

FIG. 1. Layered architecture showing a representation runtime beneath a structural computation engine and above numerical compute resources.

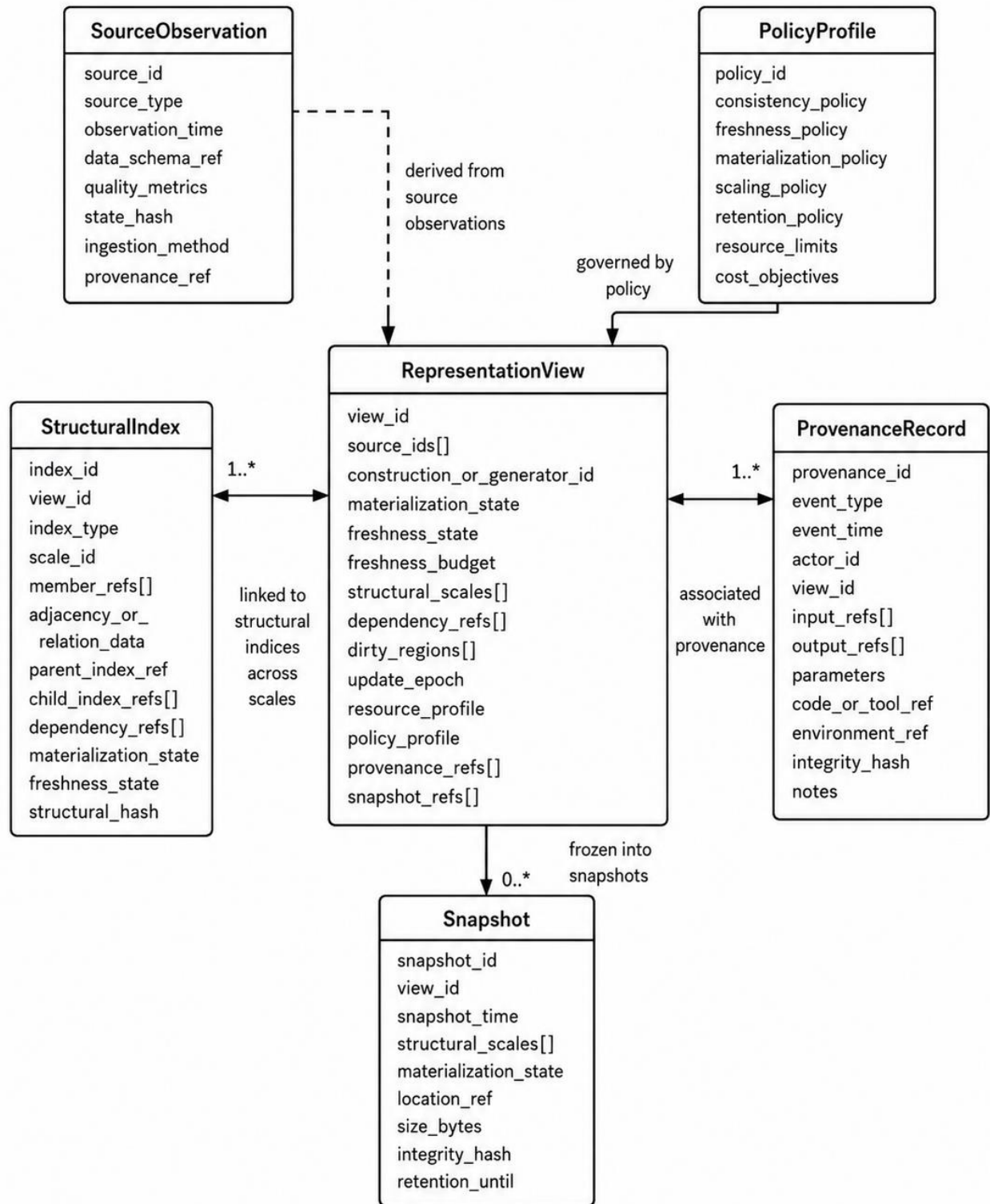


FIG. 2

FIG. 2. Representation-view object model including lifecycle, freshness, dependency, scale, and provenance metadata.

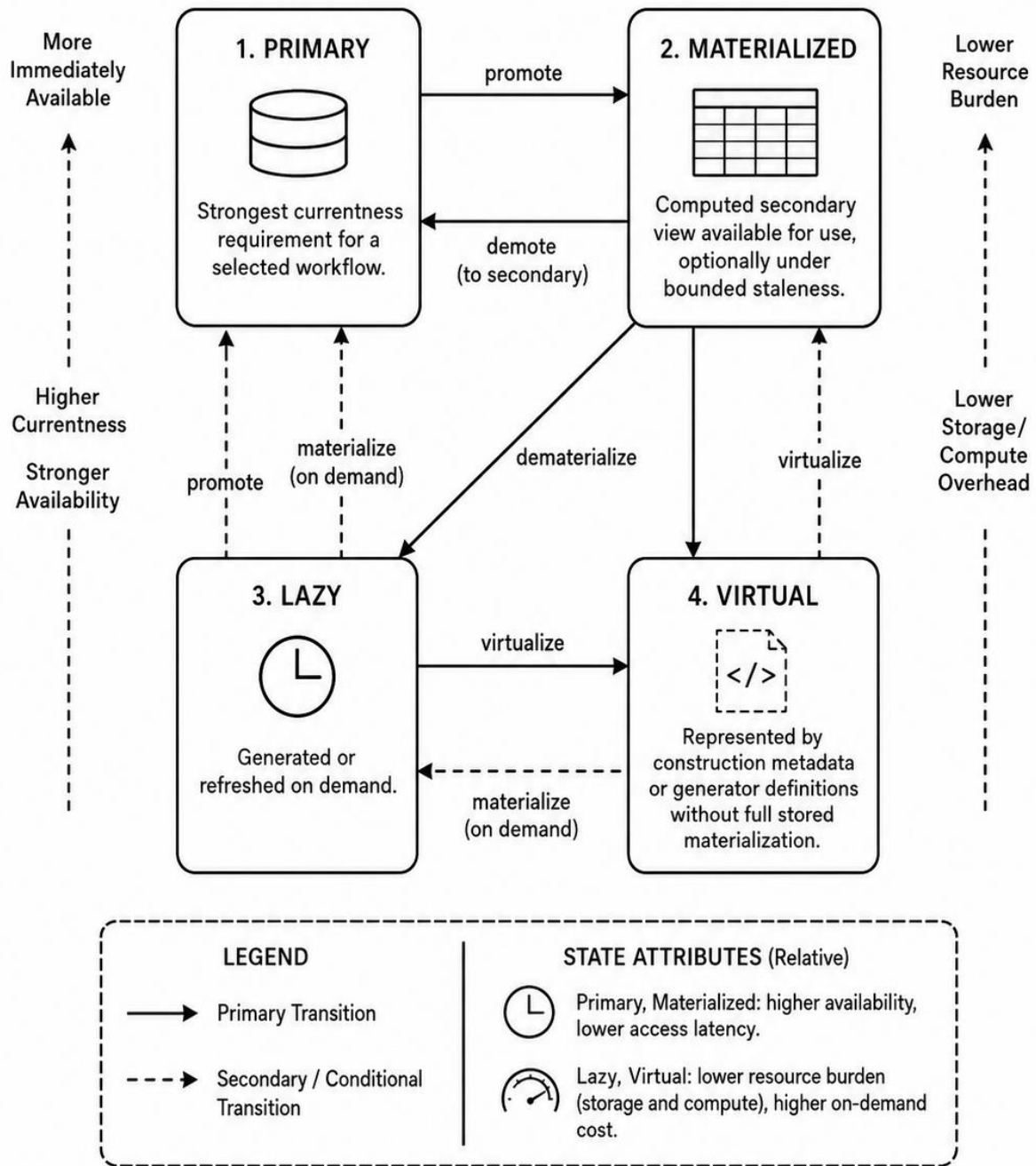


FIG. 3

FIG. 3. Materialization states including primary, materialized, lazy, and virtual.

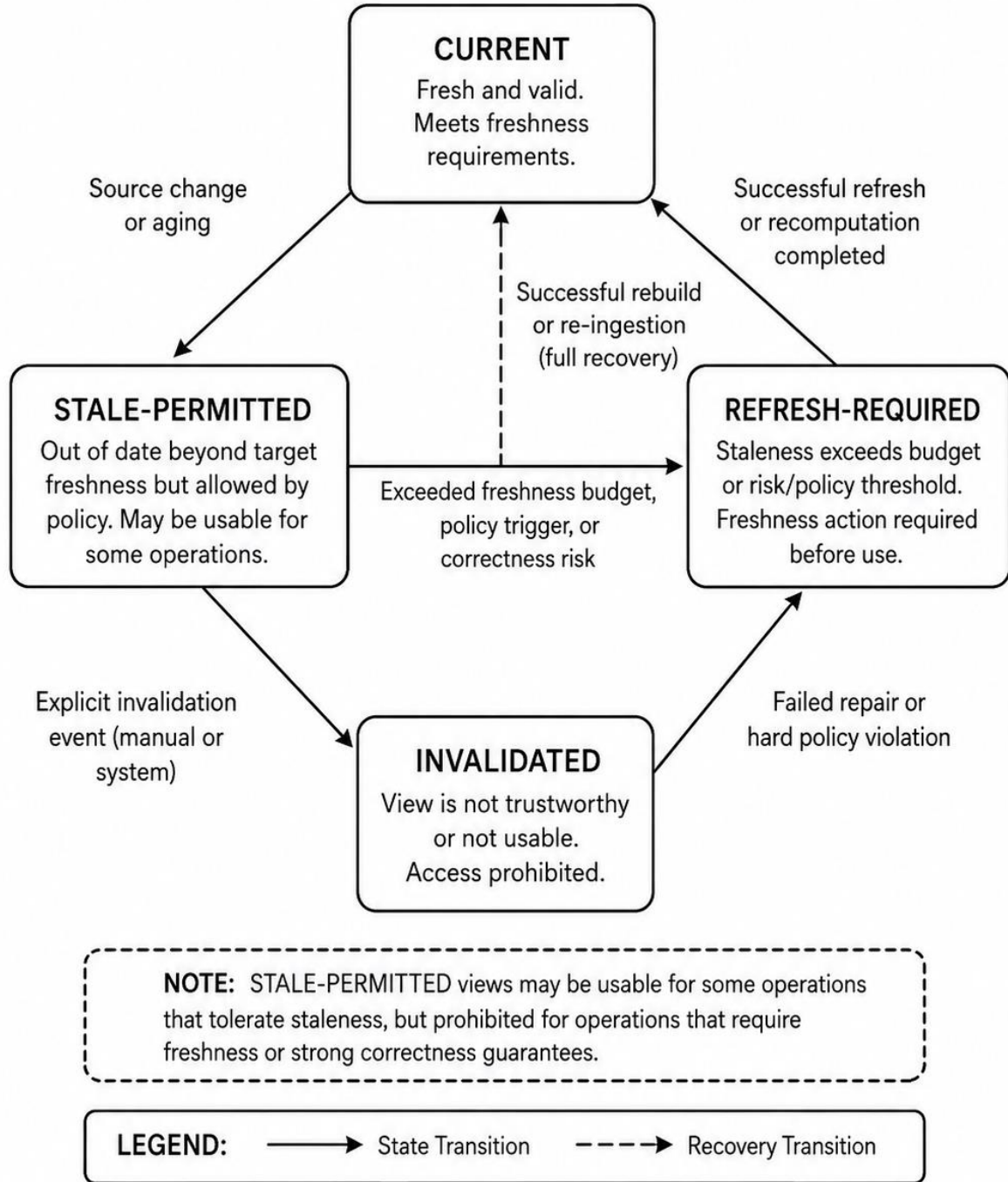


FIG. 4

FIG. 4. Freshness lifecycle including current, stale-permitted, refresh-required, and invalidated.

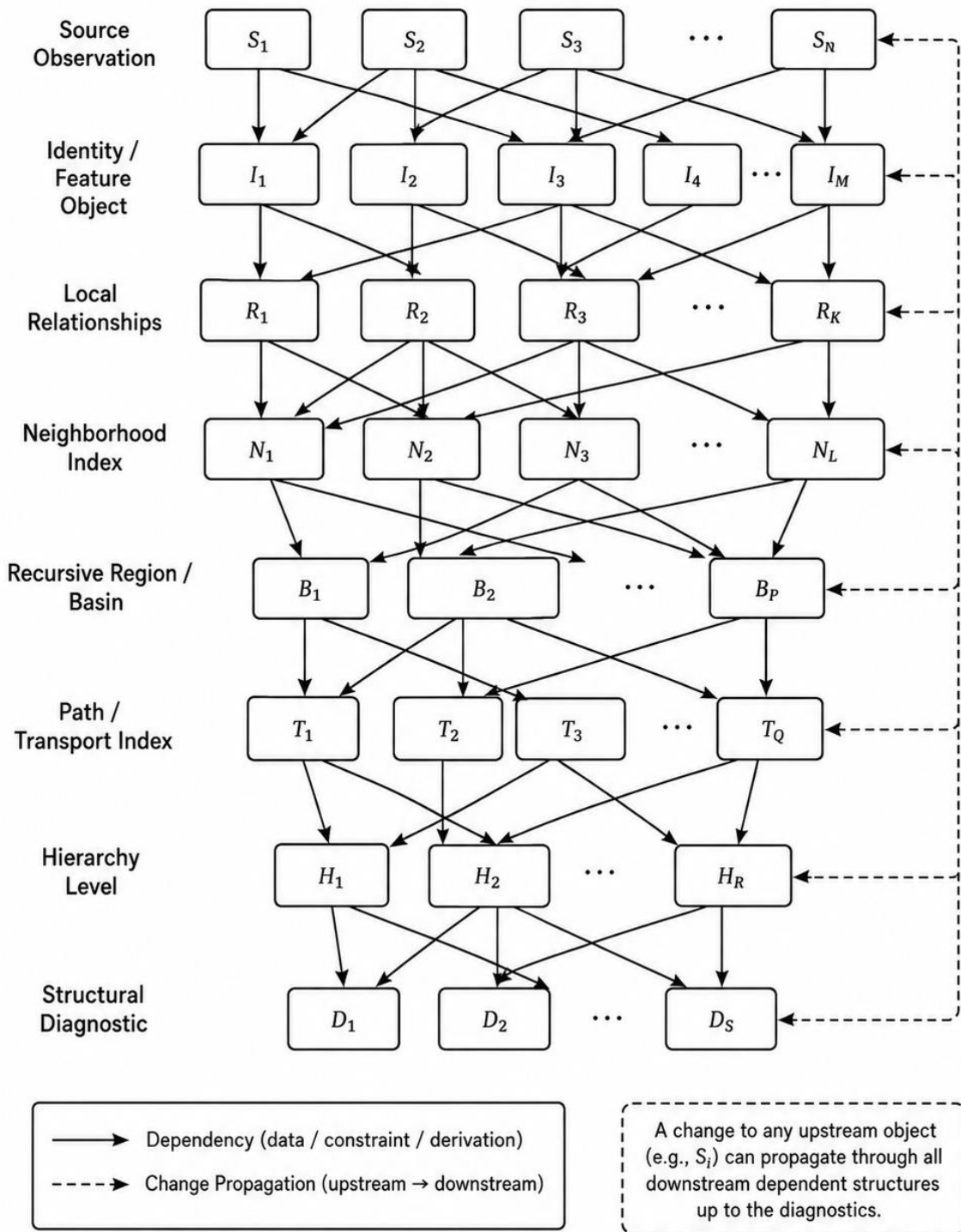


FIG. 5

FIG. 5. Dependency graph linking source observations to identities, relationships, neighborhoods, recursive regions, paths, hierarchy levels, and diagnostics.

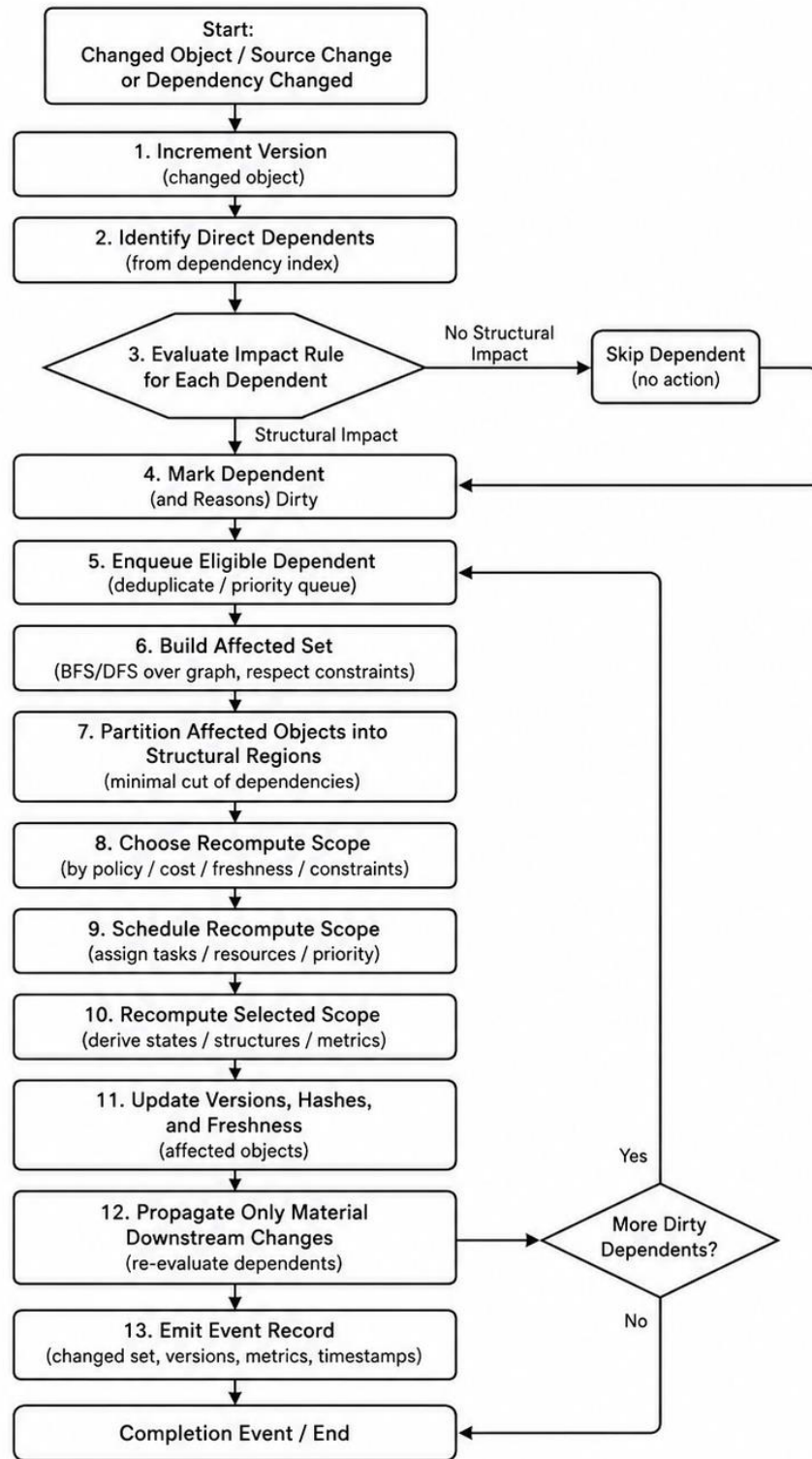


FIG. 6

FIG. 6. Event-driven structural invalidation process.

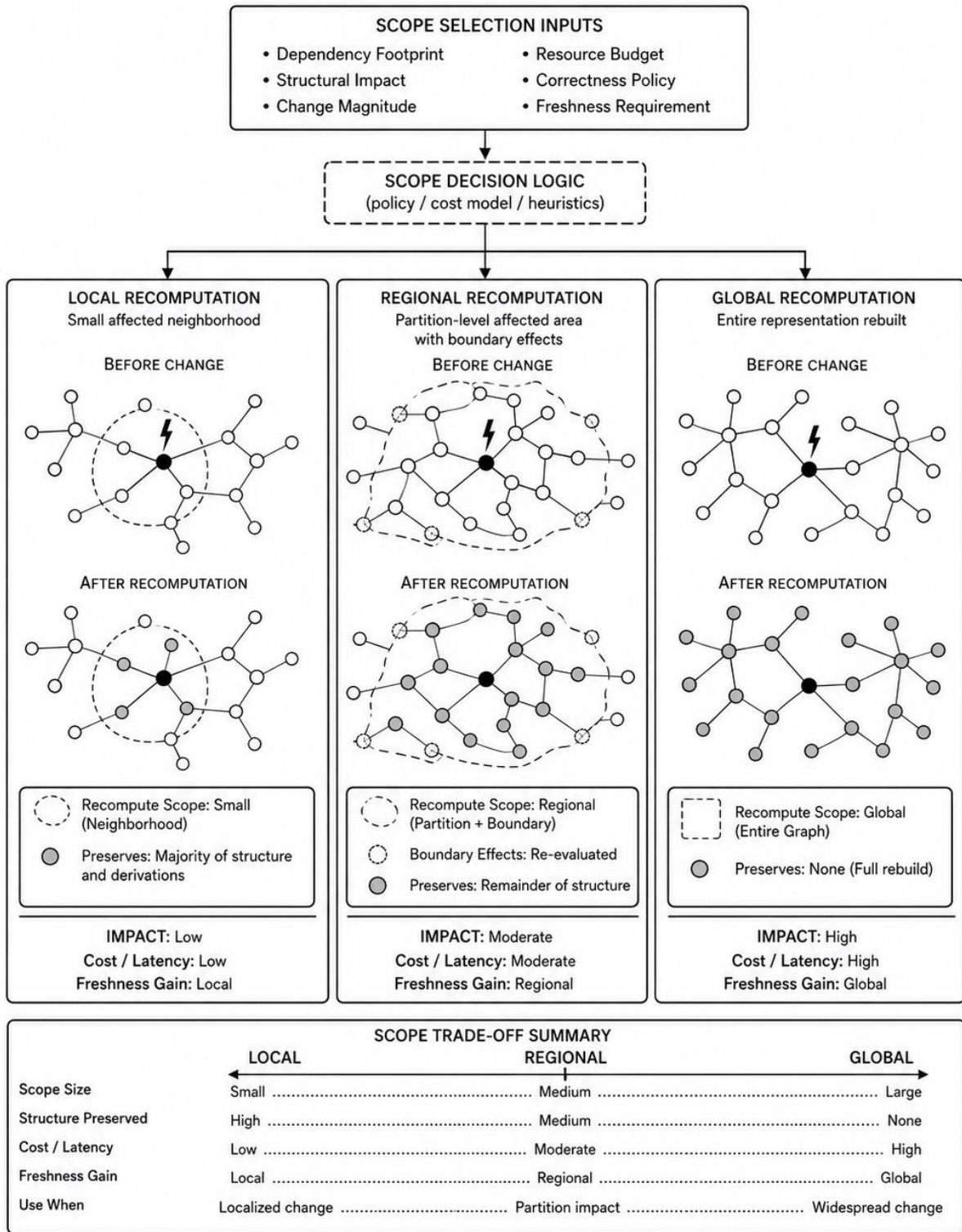


FIG. 7

FIG. 7. Local, regional, and global recomputation scopes.

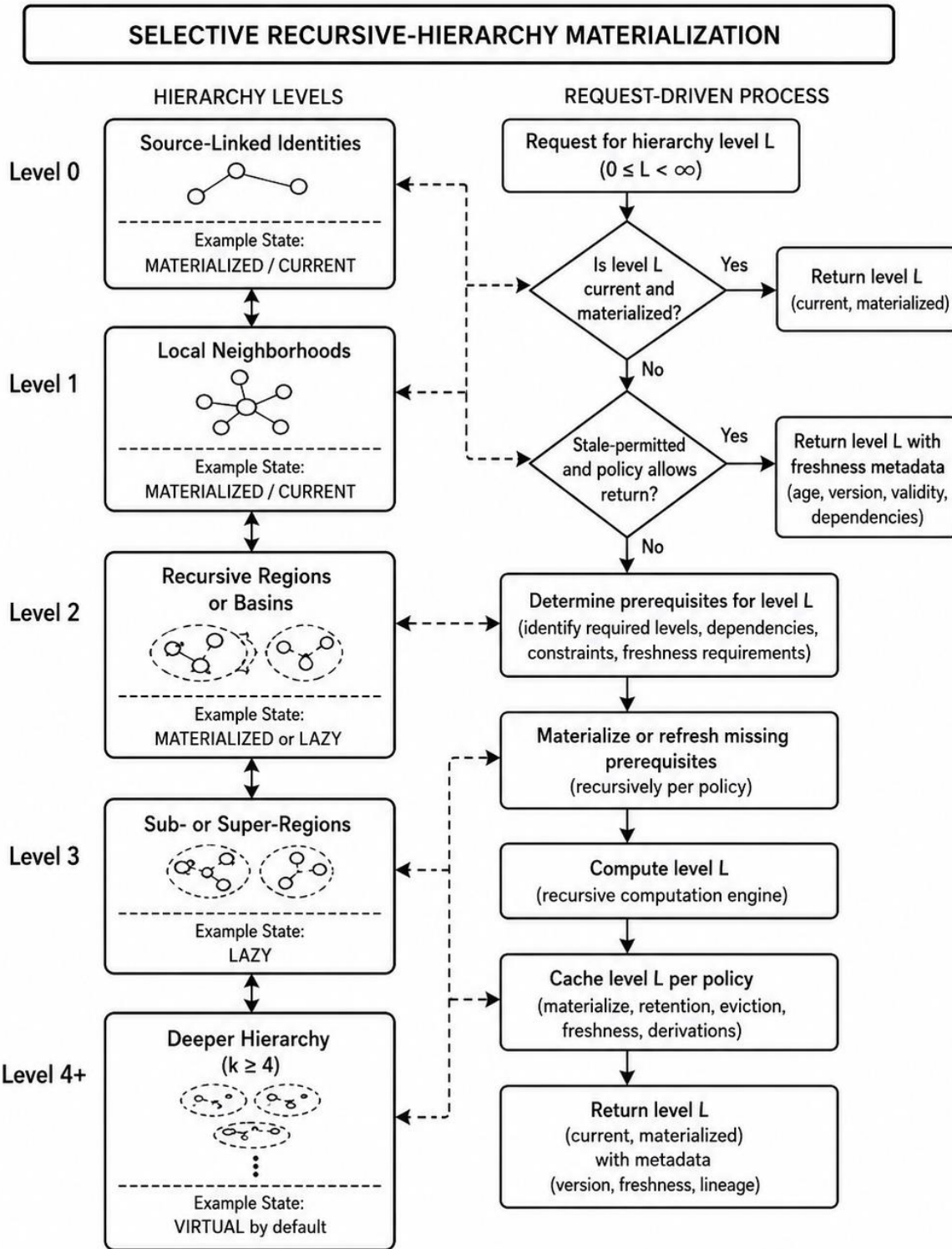


FIG. 8

FIG. 8. Selective recursive-hierarchy materialization process.

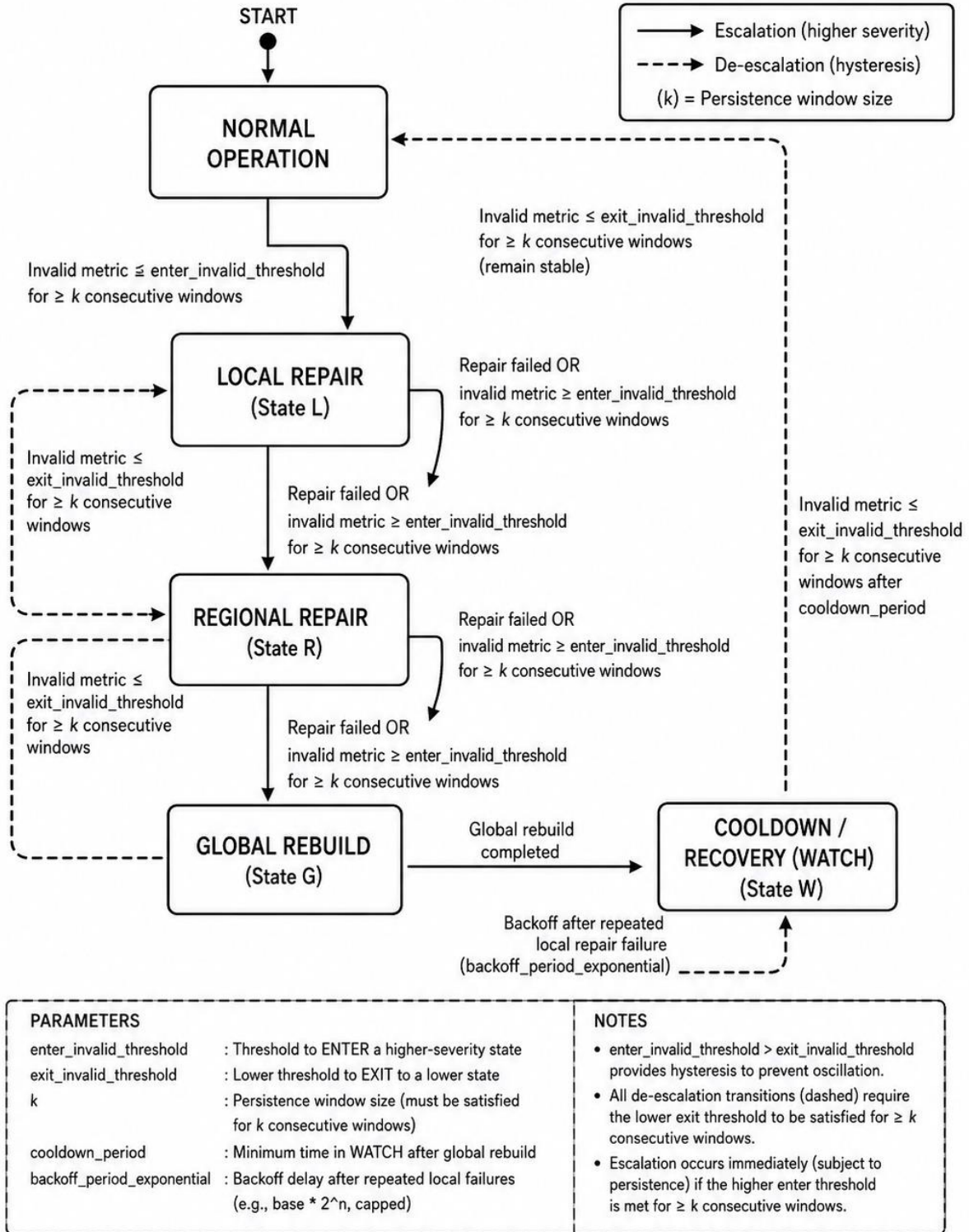


FIG. 9

FIG. 9. Anti-thrashing state machine using persistence and distinct enter/exit thresholds.

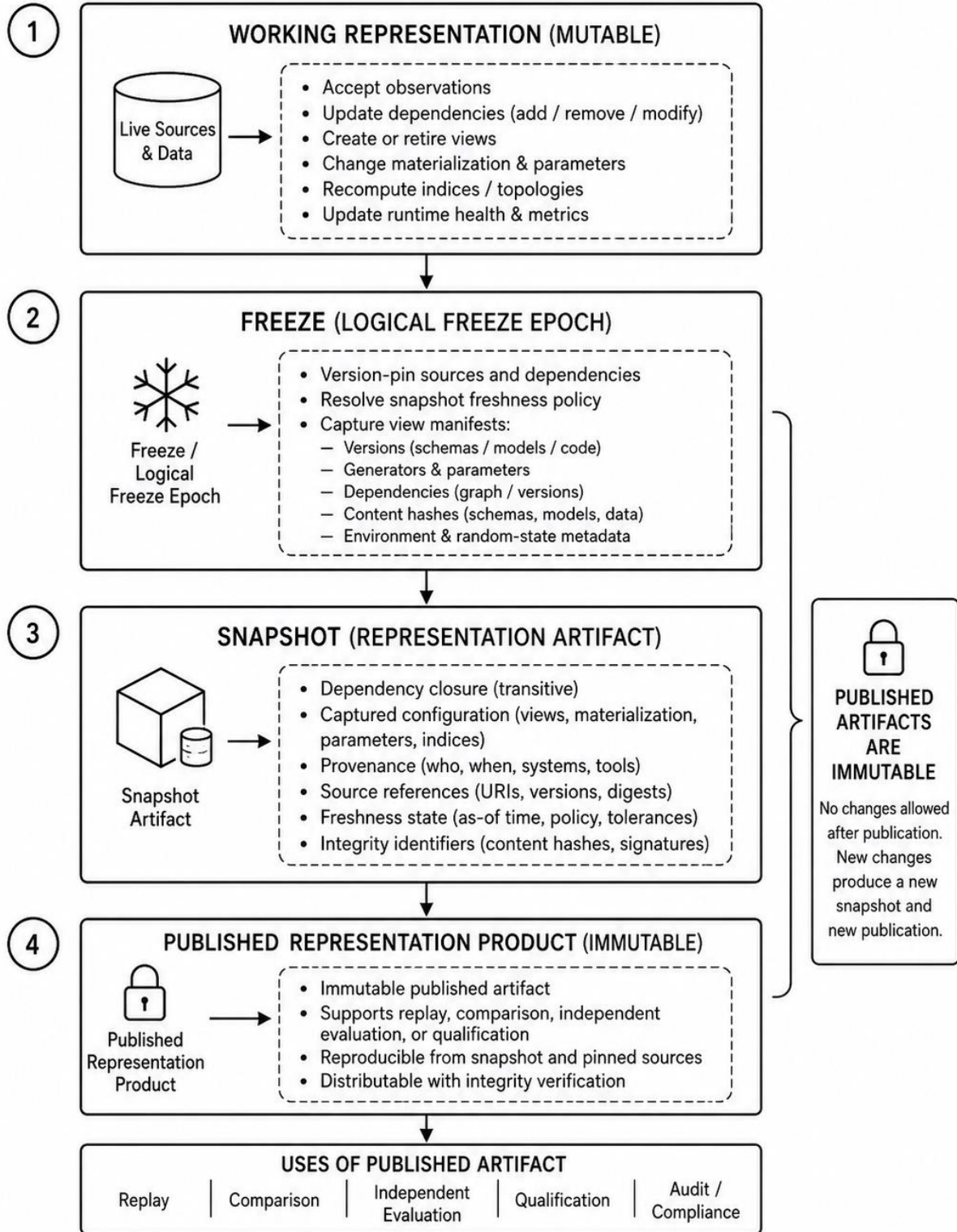


FIG. 10

FIG. 10. Working-to-snapshot-to-published lifecycle.

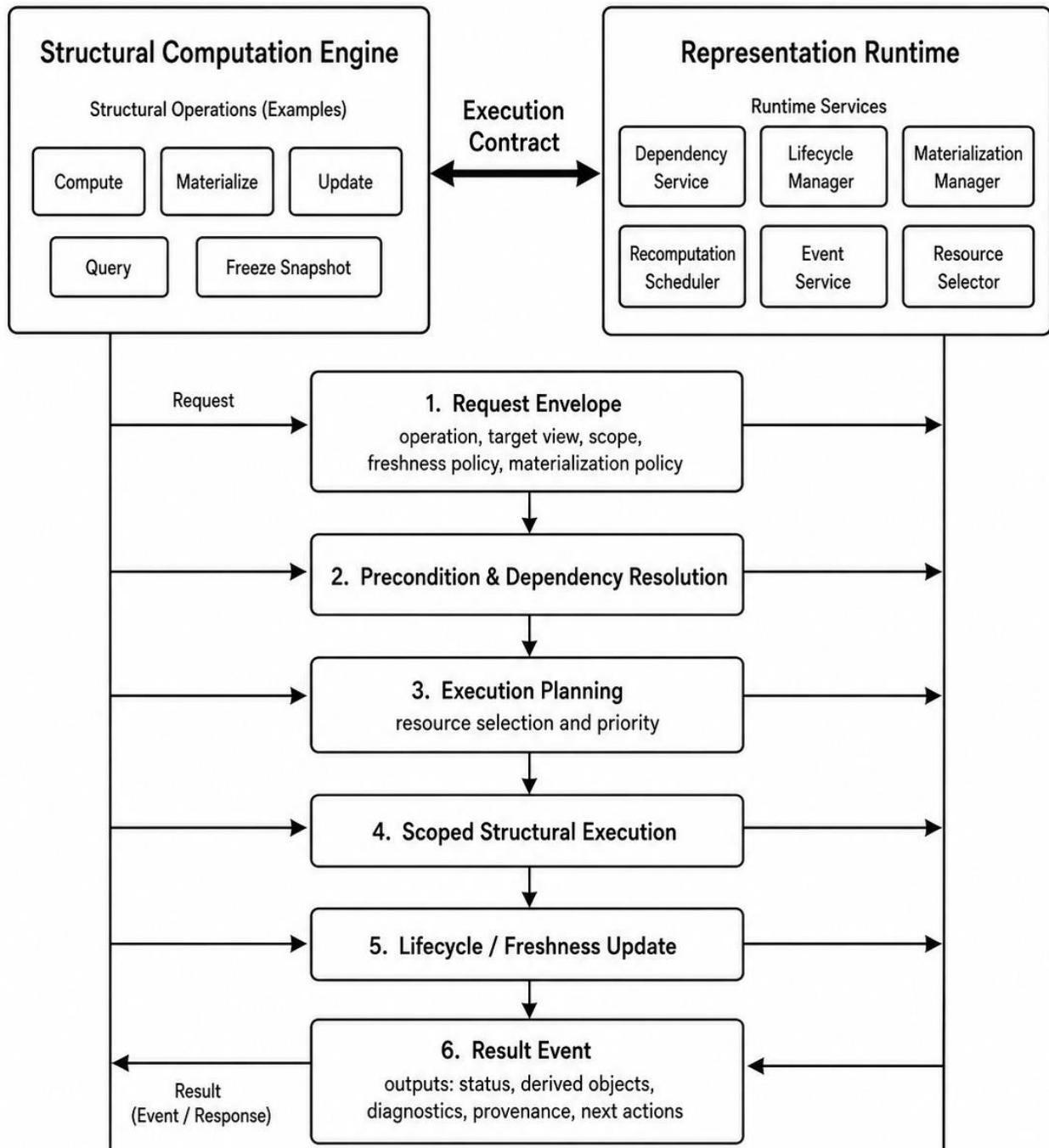


FIG. 11

FIG. 11. Execution contract between a structural computation engine and representation runtime.

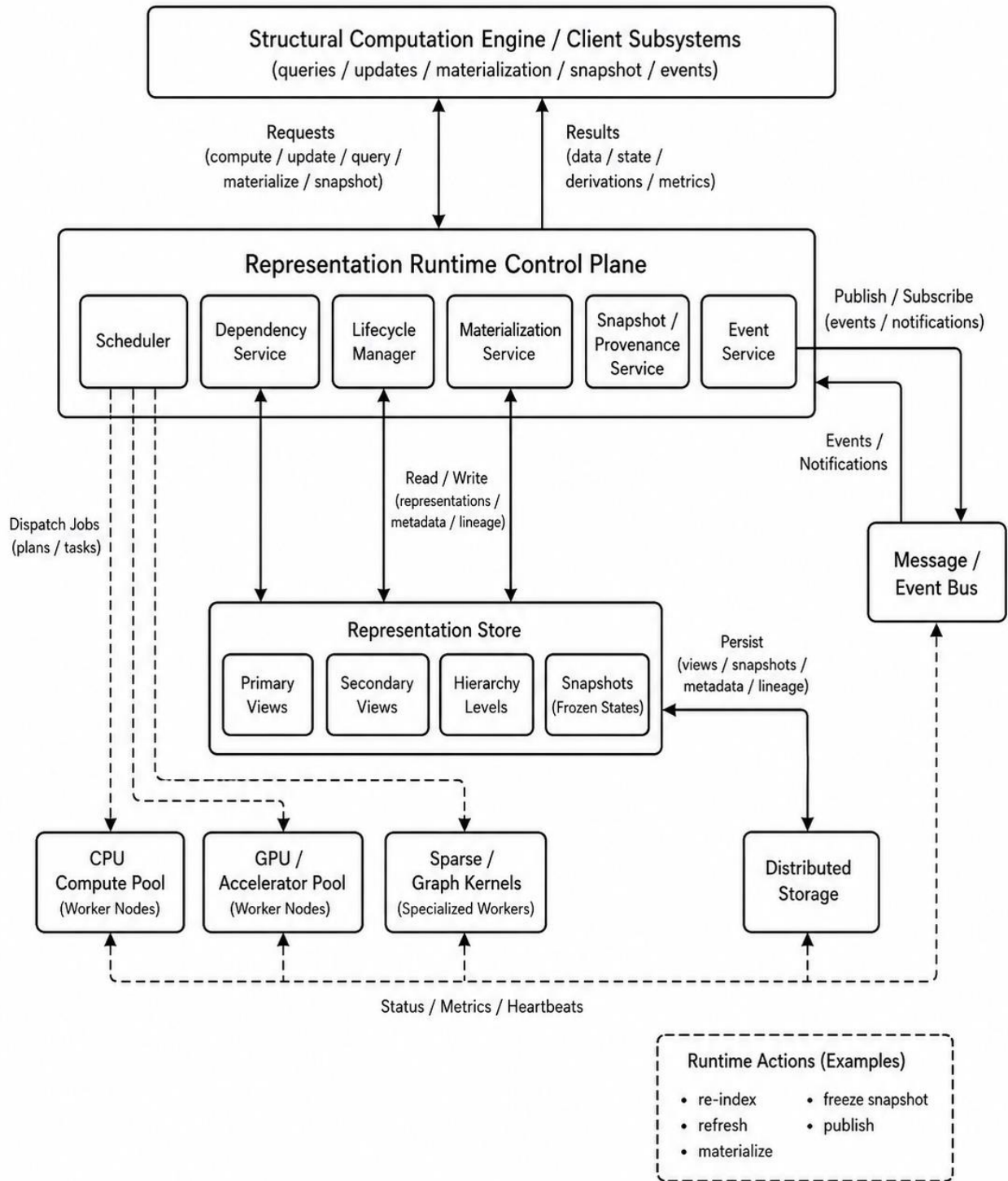


FIG. 12

FIG. 12. Distributed implementation with CPU/GPU kernels, scheduler, dependency service, representation store, and event service.

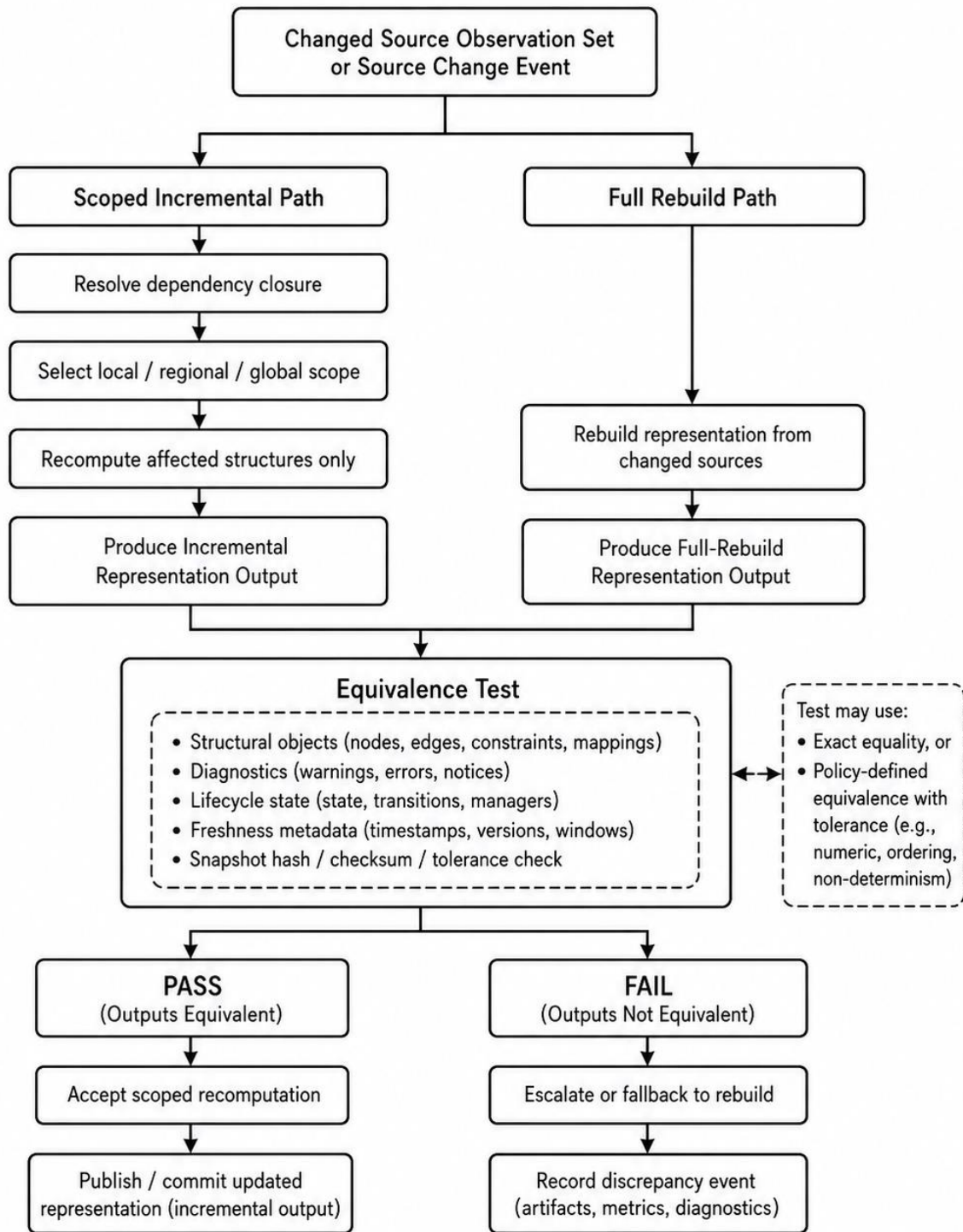


FIG. 13

FIG. 13. Incremental-update equivalence test comparing scoped recomputation with a full rebuild.

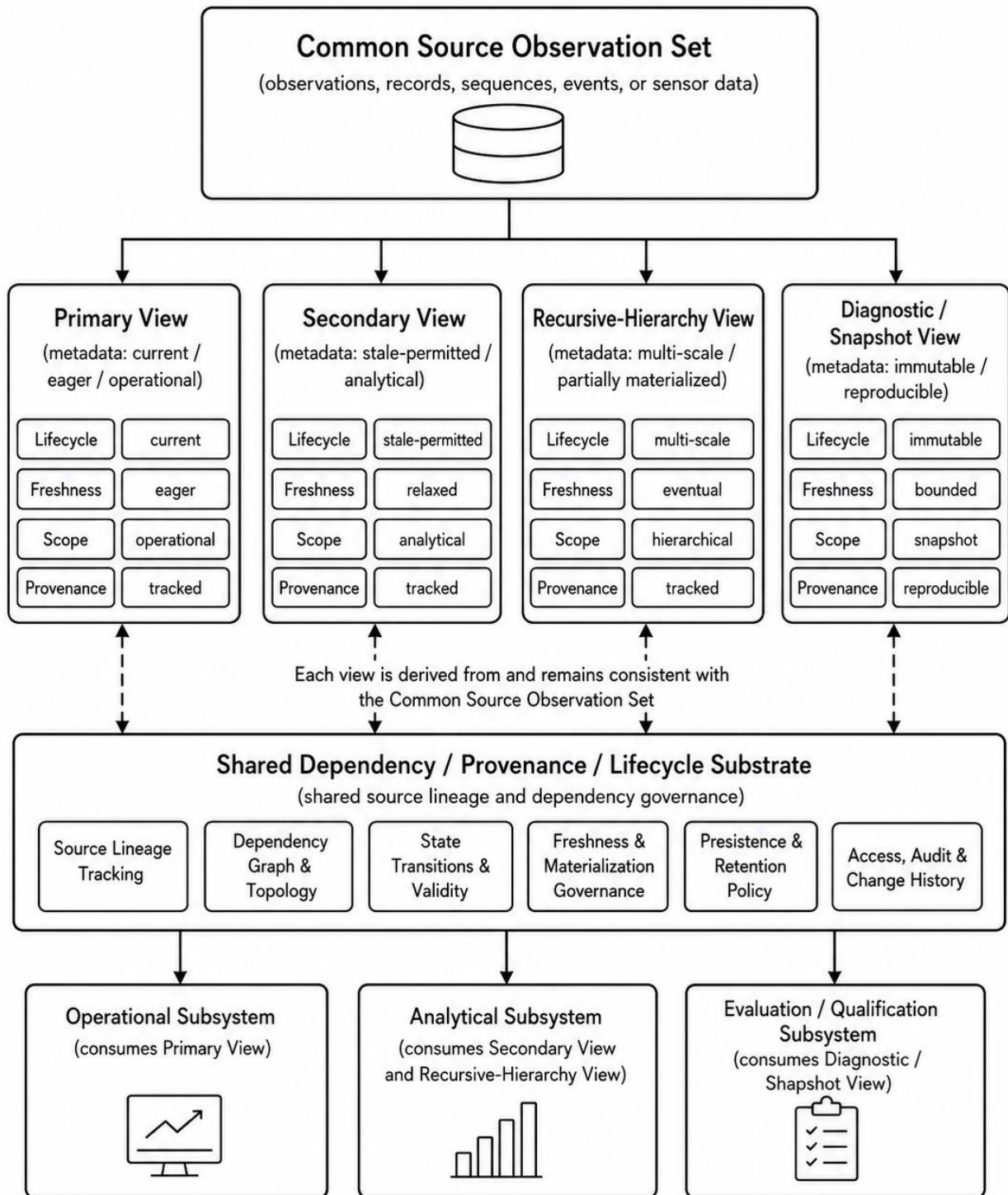


FIG. 14

FIG. 14. Multiple representation views sharing a common source observation set.

Appendix B — Example Runtime Policy Schema

```
PolicyProfile {
  primary_freshness_budget: 0 updates
  secondary_freshness_budget: 3 updates
  hierarchy_materialization: {0:eager, 1:eager, 2:adaptive, 3+:virtual}
  local_repair_radius: 2 hops
  regional_escalation: boundary_change > threshold
  global_escalation: repeated_regional_failure OR global_constraint_change
  enter_invalid_threshold: theta_enter
  exit_invalid_threshold: theta_exit
  persistence_epochs: k
  event_coalescing_window: delta_t
  snapshot_policy: STRICT_DEPENDENCY_CLOSURE
}
```

Appendix C — Focused Prior-Art Search Topics for Counsel

- incremental and dynamic graph maintenance;
- incremental view maintenance and materialized database views;
- lazy graph computation and demand-driven dataflow;
- cache coherence and dependency invalidation;
- reactive programming and self-adjusting computation;
- dynamic hierarchical clustering and community detection;
- distributed graph engines and partition-local updates;
- versioned model/feature stores and ML lineage;
- snapshot isolation, event sourcing, reproducible workflows;
- digital twins and streaming state representation maintenance;
- scientific provenance and immutable computational artifacts.